Temporary Table Issues

You said:

#Temporary Table CREATE TEMPORARY TABLE Engg(branch VARCHAR(200), students INT); select * from Engg; INSERT INTO Engg VALUES('CS',120); INSERT INTO Engg VALUES('ME',60); CREATE TEMPORARY TABLE temp1 SELECT emp_id, emp_name FROM employee; select * from temp1; DROP TEMPORARY TABLE temp1; DROP TEMPORARY TABLE Engg; select * from employee; select * from temp1;

ChatGPT said:

Your SQL is mostly correct 👍 — but there are a few **important concepts and mistakes** you need to understand about **temporary tables**.

✅ What your code is doing

1. Creating temporary table

```
SQL
CREATE TEMPORARY TABLE Engg(
    branch VARCHAR(200),
    students INT
);
```

✔️ This creates a **temporary table**
👉 Exists only for the **current session**

2. Inserting data

```
SQL
INSERT INTO Engg VALUES('CS',120);
INSERT INTO Engg VALUES('ME',60);
```

✔ Data inserted correctly

3. Creating temp table from another table

```
SQL
CREATE TEMPORARY TABLE temp1
SELECT emp_id, emp_name FROM employee;
```

✔ This copies data from `employee` into `temp1`

⚠ BUT: This will **fail if `employee` table doesn't exist**

4. Dropping temp tables

```
SQL
DROP TEMPORARY TABLE temp1;
DROP TEMPORARY TABLE Engg;
```

✔ Tables are deleted manually

❌ Main Problem in your last queries

```
SQL
select * from temp1;
```

👉 This will give ERROR ❌

Why?

Because:

```
SQL
DROP TEMPORARY TABLE temp1;
```

👉 You already deleted it

⚠️ Important Rules of Temporary Tables

1. Session-based

- Temporary tables exist **only in current connection**
- If you restart MySQL → table gone automatically

2. After DROP → cannot access

```
SQL
DROP TEMPORARY TABLE temp1;
SELECT * FROM temp1;   -- ❌ ERROR
```

3. If `employee` doesn't exist

You will get:

```
Error Code: 1146. Table 'employee' doesn't exist
```

✅ Correct Flow (Safe Version)

```
SQL
-- Create temp table
CREATE TEMPORARY TABLE Engg(
    branch VARCHAR(200),
    students INT
);

INSERT INTO Engg VALUES('CS',120), ('ME',60);

SELECT * FROM Engg;

-- Create temp from employee (only if employee exists)
CREATE TEMPORARY TABLE temp1
SELECT emp_id, emp_name FROM employee;

SELECT * FROM temp1;

-- Drop after use
DROP TEMPORARY TABLE temp1;
DROP TEMPORARY TABLE Engg;
```

💡 Pro Tip (Interview Point)

👉 Temporary tables are used for:

- Intermediate calculations
- Data transformation
- Avoid modifying original tables
- Improving query performance

🔥 Your mistake summary

Issue	Reason
`SELECT * FROM temp1` after DROP	Table no longer exists
Possible error in `employee`	Table may not exist

If you want, I can give you:
👉 Real-world examples of temporary tables
👉 OR interview questions on this topic
👉 OR difference between temp table vs CTE vs view

Just tell me 👍

You said:

#Temporary Table CREATE TEMPORARY TABLE Engg(branch VARCHAR(200), students INT); select * from Engg; INSERT INTO Engg VALUES('CS',120); INSERT INTO Engg VALUES('ME',60); CREATE TEMPORARY TABLE temp1 SELECT emp_id, emp_name FROM employee; select * from temp1; DROP TEMPORARY TABLE temp1; DROP TEMPORARY TABLE Engg; select * from employee; select * from temp1; explain each line

ChatGPT said:

Got it — let's go **line by line** and understand exactly what each statement does 👇

🔹 1. Create Temporary Table

```
SQL
CREATE TEMPORARY TABLE Engg(
branch VARCHAR(200),
students INT);
```

✅ Creates a **temporary table named** `Engg`

- `branch` → stores branch names (like CS, ME)
- `students` → stores number of students
- Exists **only in current session**

🔹 2. View empty table

```
SQL
select * from Engg;
```

✅ Displays all records from `Engg`

👉 Output: **Empty table** (no data yet)

🔹 3. Insert first record

```
SQL
INSERT INTO Engg
VALUES('CS',120);
```

✅ Adds one row:

```
CS | 120
```

🔹 4. Insert second record

```
SQL
INSERT INTO Engg
VALUES('ME',60);
```

✅ Adds another row:

```
ME | 60
```

🔹 5. Create another temporary table from existing table

```
SQL
CREATE TEMPORARY TABLE temp1
SELECT emp_id, emp_name FROM employee;
```

✅ Creates `temp1` and copies data from `employee`

👉 Important:

- `temp1` structure is automatically created
- Only `emp_id` and `emp_name` are copied

⚠️ Condition:

- `employee` table **must exist**, otherwise error

🔹 6. View temp1 data

```
SQL
select * from temp1;
```

✅ Shows copied data from `employee`

🔹 7. Drop temp1

```
SQL
DROP TEMPORARY TABLE temp1;
```

❌ Deletes `temp1` permanently (in current session)

🔹 8. Drop Engg

```
SQL
DROP TEMPORARY TABLE Engg;
```

❌ Deletes `Engg` table

🔹 9. View original table

```
SQL
select * from employee;
```

✅ Displays data from original `employee` table

✔️ This works because:

- `employee` is **not temporary**
- Still exists in database

🔹 10. Try to access deleted temp table

```
SQL
select * from temp1;
```

❌ ERROR ❌

👉 Reason:

- You already ran:

  ```
  SQL
  DROP TEMPORARY TABLE temp1;
  ```

- So `temp1` no longer exists

🚨 Final Output Summary

Step	Result
Create Engg	✅ Success

Step	Result
Insert data	✅ Success
Create temp1	✅ (if employee exists)
Select temp1	✅
Drop temp1	❌ Deleted
Select temp1 again	❌ ERROR

🔥 Key Concept (Very Important)

👉 Temporary Tables:

- Exist **only during session**
- Auto-deleted when session ends
- Cannot be used after `DROP`

💡 Simple Analogy

Think of temporary table like:
👉 A **rough notebook page**

- You write data → use it
- Tear the page → gone forever ❌

If you want next:
👉 Difference between **Temporary Table vs View vs CTE** (very important for interviews)
👉 OR tricky SQL questions based on this

Just tell me 👍

You said:

select * from employee order by score desc limit 1; select * from employee where score=(select max(score) from employee); select * from employee where score=(select min(score) from employee); select * from employee where score in (5, 9.3); select * from employee where score=(select max(score) from employee) union select * from employee where score=(select min(score) from

employee); #subquery2 select * from employee where score in ((select min(score) from employee), (select max(score) from employee)); #subquery3 select * from employee where job_domain= (select high_job_domain.job_domain from (select job_domain, count(*) from employee group by job_domain limit 1) as high_job_domain); select job_domain, count(*) from employee group by job_domain limit 1; #ANY & ALL select * from employee where score > all(select score from employee where job_domain="sales"); select * from employee where score > any(select score from employee where job_domain="sales"); select * from employee where score>(select max(score) from employee where job_domain="sales"); select * from employee where score>(select min(score) from employee where job_domain="sales"); #co-relatedsubquery select emp_id, emp_name, (select salary from income where emp_id = employee.emp_id) as salary from employee order by salary desc; #CTE with study1 as (select employee.emp_id, emp_name, job_domain, salary from employee inner join income using (emp_id)) select emp_name, salary from study1 where salary>3; select emp_name, salary from study1 where salary>5; explain each line code easily

ChatGPT said:

Great — this is a **full set of important SQL interview queries** 🔥
I'll explain **each line in simple language** so you can *understand + remember*.

🔹 1. Highest score (Method 1)

```
SQL
select * from employee order by score desc limit 1;
```

✅ Sorts all employees by `score` in **descending order**
✅ `limit 1` → returns **top 1 row**

👉 Gives employee with **highest score**

🔹 2. Highest score (Method 2)

```
SQL
select * from employee where score=(select max(score) from
employee);
```

✅ Subquery finds **maximum score**
✅ Outer query returns all employees having that score

👉 Useful when **multiple people have same highest score**

🔹 3. Lowest score

```
SQL
select * from employee where score=(select min(score) from
employee);
```

✅ Finds employee(s) with **minimum score**

🔹 4. Using IN

```
SQL
select * from employee where score in (5, 9.3);
```

✅ Returns employees whose score is **either 5 or 9.3**

🔹 5. Highest + Lowest using UNION

```
SQL
select * from employee where score=(select max(score) from
employee)
union
select * from employee where score=(select min(score) from
employee);
```

✅ Combines:

- highest score employees
- lowest score employees

👉 `UNION` removes duplicates

🔹 6. Same using IN + subquery

```
SQL
select * from employee where score in (
    (select min(score) from employee),
    (select max(score) from employee)
);
```

✅ Cleaner way to get **min + max together**

🔹 7. Subquery with GROUP BY (Important)

```
SQL
select * from employee where job_domain=
(select high_job_domain.job_domain from
    (select job_domain, count(*) from employee group by
job_domain limit 1) as high_job_domain);
```

👉 Step-by-step:

1. Inner query:

   ```
   SQL
   select job_domain, count(*) from employee group by
   job_domain limit 1
   ```

   → Groups employees by domain (IT, Sales, etc.)
   → Counts employees in each
   → `limit 1` → picks **one domain** (⚠️ not necessarily highest!)

2. Outer query:
   → Gets all employees from that domain

⚠️ Problem:

- No `ORDER BY` → result is random

🔹 8. Simpler version

```
SQL
select job_domain, count(*) from employee group by job_domain
limit 1;
```

✅ Shows one domain with count
⚠️ Again, **not guaranteed highest**

🔹 9. ANY & ALL (Very Important Concept)

ALL

```
SQL
select * from employee where score > all(select score from
employee where job_domain="sales");
```

✅ Employee score must be **greater than ALL sales scores**

👉 Equivalent to:

```
SQL
score > max(sales scores)
```

ANY

```
SQL
select * from employee where score > any(select score from
employee where job_domain="sales");
```

✅ Employee score must be **greater than at least ONE sales score**

👉 Equivalent to:

```
SQL
score > min(sales scores)
```

Same logic simplified

```
SQL
select * from employee where score>(select max(score) from
employee where job_domain="sales");
```

✔️ Same as `> ALL`

```
SQL
select * from employee where score>(select min(score) from
employee where job_domain="sales");
```

✔️ Same as `> ANY`

🔹 10. Correlated Subquery

```
SQL
select emp_id, emp_name,
(select salary from income where emp_id = employee.emp_id) as
salary
from employee
order by salary desc;
```

✅ For each employee:

- Subquery runs again and again
- Finds matching salary from `income` table

👉 Adds salary column to result

⚠️ Called **correlated** because:

- Inner query depends on outer query (`employee.emp_id`)

🔹 11. CTE (Common Table Expression)

```
SQL
with study1 as (
    select employee.emp_id, emp_name, job_domain, salary
    from employee
    inner join income
    using (emp_id)
)
select emp_name, salary from study1 where salary>3;
```

✅ Step-by-step:

1. `study1` is a **temporary result set (like virtual table)**

2. It joins:
    - `employee` + `income`
3. Final query:
   → shows employees with salary > 3

🔹 12. Error in your last query

```
SQL
select emp_name, salary from study1 where salary>5;
```

❌ ERROR ❌

👉 Why?

- `study1` exists only inside previous query
- CTE is **not permanent**

🔥 Key Concepts Summary

Concept	Meaning
`LIMIT`	Restrict rows
`MAX/MIN`	Find highest/lowest
`IN`	Match multiple values
`UNION`	Combine results
`ANY`	Compare with at least one
`ALL`	Compare with all values
Correlated Subquery	Runs per row
CTE	Temporary query result

💡 Interview Tip

👉 Best way to get highest score:

```
SQL
select * from employee where score=(select max(score) from
employee);
```

👉 Best way to join salary:

```
SQL
SELECT e.emp_name, i.salary
FROM employee e
JOIN income i ON e.emp_id = i.emp_id;
```

✔️ Faster than correlated subquery

If you want next level 🚀
I can give:

- 🔥 20 SQL interview questions (Amazon/Microsoft level)
- OR convert all this into **notes PDF style**
- OR give **practice problems with solutions**

Just tell me 👍

ChatGPT can make mistakes. Check important info. See Cookie Preferences.